

作业管理系统过程剖析

1 开发与调试概述

本系统开发过程主要围绕三类问题展开：一是 Web 表单输入不符合预期时，后端路由是否会直接崩溃；二是 Python 文件在修改过程中是否存在语法或缩进错误；三是修复后是否有测试用例或命令行检查保证问题不再出现。为了避免只描述结果，本报告保留了两个真实调试案例：一个来自 Flask 路由业务逻辑，一个来自 Python 编译检查阶段。

调试时使用的主要命令如下：

```
.venv\Scripts\python -m unittest discover -s tests -v  
.venv\Scripts\python -m py_compile docs\bug_repro\compile_indent_after.py
```

这些命令分别用于运行系统工作流测试和检查 Python 源文件是否能通过编译。下面按问题出现、原因分析、修复方法、修复后验证四个角度记录。

2 问题一：发布作业时课程编号异常导致服务器错误

2.1 问题现象

在教师发布作业功能调试时，我发现后端接口对表单中的课程编号缺少异常处理。正常页面上的课程下拉框一般只会提交数字编号，但如果浏览器重复提交旧页面、表单被异常修改，或前端传来的 `course_id` 不是数字，后端会直接执行 `int()` 转换并抛出异常，页面显示 500 错误。

Internal Server Error

The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.

图 1：修复前课程编号异常导致 500 错误

2.2 排查过程

根据错误日志继续向下追踪，异常位置定位在 `assignment_system/routes.py` 的教师发布作业路由中。原代码直接写成：

```
course_id = int(request.form.get("course_id", 0))
```

这行代码默认表单中的课程编号一定可以转成整数。实际测试时提交 `course_id=abc`，程序会抛出：

```
ValueError: invalid literal for int() with base 10: 'abc'
```

这个问题属于典型的表单输入校验不完整。虽然正常用户通过下拉框操作时不容易遇到，但后端不能完全依赖前端控件，仍然需要对收到的数据做校验。

2.3 修复方法

修复时在课程编号转换外层增加 `try/except`。当课程编号缺失或不是数字时，不再让 Flask 返回 500，而是向用户显示提示并回到教师工作台。

```
try:
    course_id = int(request.form.get("course_id", 0))
except (TypeError, ValueError):
    flash("请选择有效课程。")
    return redirect(url_for("teacher_dashboard"))
```

这样处理后，后端对异常表单数据有了明确兜底，页面行为也和普通业务校验保持一致。

2.4 修复后效果

使用同样的异常表单再次提交后，系统不再出现 500 错误，而是回到教师工作台，并在顶部显示“请选择有效课程。”提示。

The screenshot shows the '教师工作台' (Teacher's Workbench) interface. At the top, there is a blue banner with the text '请选择有效课程。' (Please select an effective course.). Below this, the interface is divided into three main sections:

- 开设课程 (Create Course):** Includes a '课程名称' (Course Name) input field, a '课程说明' (Course Description) text area, and a '创建课程' (Create Course) button.
- 布置作业 (Assign Homework):** Includes a '选择课程' (Select Course) dropdown menu (currently showing 'Python 程序设计基础'), a '作业标题' (Assignment Title) input field, a '截止时间' (Deadline) input field (format: yyyy/mm/日 --:--), and a '作业要求' (Assignment Requirements) text area. A '发布作业' (Publish Homework) button is at the bottom.
- 我的课程 (My Courses):** A section titled 'Python 程序设计基础' (Python Programming Fundamentals) with a description '面向 Python 基础语法、文件处理和数据库应用的课程。' (Course for Python basic syntax, file processing, and database applications) and '1 名学生' (1 student).

图 2: 修复后显示有效课程提示

2.5 自动化验证

为了避免以后修改时再次出现同类问题，我补充了单元测试 `test_invalid_assignment_course_id_shows_message`。测试过程先登录教师账号，再向发布作业接口提交异常的 `course_id=abc`，期望返回正常页面并显示提示信息。

修复前，新增测试会失败，错误位置指向 `course_id = int(...)`；修复后，全部测试通过：

```
test_admin_teacher_student_workflow ... ok
test_invalid_assignment_course_id_shows_message ... ok
test_pending_teacher_cannot_login_before_approval ... ok
```

```
Ran 3 tests in 2.149s
```

```
OK
```

3 问题二: Python 编译检查发现缩进错误

3.1 问题现象

Python 对缩进非常敏感。开发过程中,如果在循环、判断、函数等代码块下面少缩进一层,程序在真正运行前就会失败。为了在提交前提前发现这种问题,我使用 `py_compile` 对单个 Python 文件进行编译检查。修复前临时复现文件中的关键代码如下:

```
def total_score(items):  
    total = 0  
    for item in items:  
total += item["score"]  
    return total
```

其中 `total += item["score"]` 应该属于 `for` 循环体,但实际没有缩进到循环内部。运行编译检查后,终端直接报出 `IndentationError`。

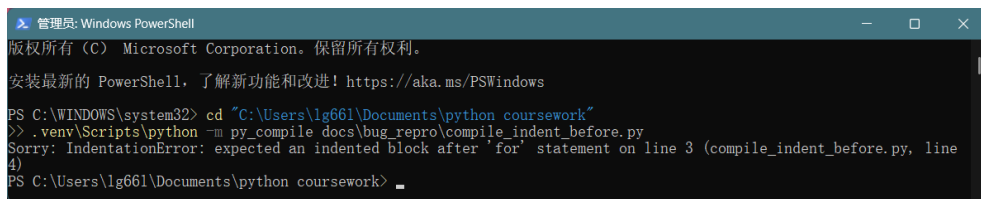


图 3: 修复前 `py_compile` 检查发现缩进错误

3.2 原因分析

这个错误不是业务逻辑错误,而是 Python 源代码在解释执行前的语法结构错误。它的特点是:

- 浏览器页面还没有机会打开,程序就已经无法通过编译。
- 错误位置通常会指向代码块后的第一行异常缩进位置。
- 如果只靠肉眼检查,长文件中很容易漏掉一行缩进。

因此在调试这类问题时,先用 `py_compile` 做快速检查比直接启动 Web 服务更清楚。它可以把问题限定在某个 Python 文件内部,避免把语法错误误判成 Flask 或数据库问题。

3.3 修复方法

修复方式是把累加语句缩进到 for 循环体内部，使代码块结构和程序意图一致：

```
def total_score(items):  
    total = 0  
    for item in items:  
        total += item["score"]  
    return total
```

修复后再次运行：

```
.venv\Scripts\python -m py_compile docs\bug_repro\compile_indent_after.py
```

命令可以正常结束，并输出手动补充的通过提示。

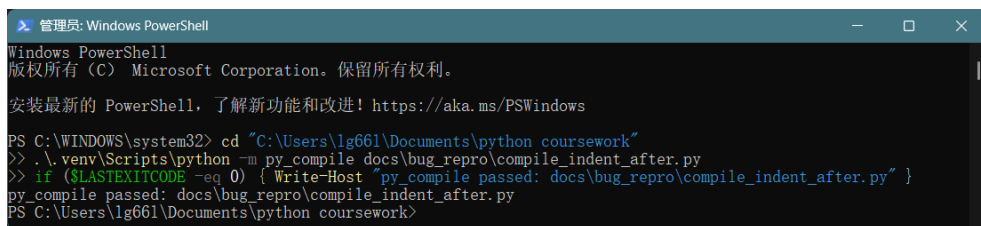


图 4：修复后 py_compile 检查通过

4 调试经验总结

两个问题分别对应了开发过程中的两层质量检查。第一个问题发生在 Web 运行阶段，需要通过异常请求和单元测试发现；第二个问题发生在 Python 编译阶段，可以在运行服务前通过 py_compile 提前发现。对课程作业项目来说，这两类检查都很有用。

问题类型	表现	处理方式
表单输入异常	页面返回 500，后端日志出现 ValueError	增加 try/except 和业务提示，并补充单元测试
Python 缩进错误	py_compile 报 IndentationError	调整代码块缩进，再次运行编译检查
回归验证不足	修复后无法确认是否会复发	将异常输入场景写入 tests 目录下的测试用例

通过这次调试可以看出，系统开发不能只关注“正常流程能跑通”。后端接口应当假设输入可能不合法，Python 文件也应在提交前先通过编译检查。最终修复后，系统在异常输入下不再崩溃，代码也通过了单元测试和编译检查，交付稳定性更高。